

Developer's Guide to Workshop 1: Realization and Practical Exercises

This document is your hands-on companion for the workshop. Follow these steps to complete all the practical exercises and achieve the workshop's deliverables.

Step 1: Environment Setup & Verification

The first task is to ensure your local machine is ready for Flutter development.

1. **Install Flutter:** Follow the official Flutter installation guide for your operating system.
 2. **Verify Your Installation:** Open your terminal and run the following command. It will check for any missing dependencies.

```
Bash  
flutter doctor
```

 - **Troubleshooting:** If flutter doctor reports any issues (e.g., missing Android SDK, cmdline-tools component, or Xcode dependencies), follow the instructions in the terminal to resolve them.
 3. **Setup an IDE:** Install Visual Studio Code or Android Studio.
 - **VS Code:** Install the official **Flutter** and **Dart** extensions from the marketplace.
 - **Android Studio:** Install the Flutter and Dart plugins via Settings/Preferences -> Plugins.
 4. **Launch a Device/Emulator:**
 - **Android Studio:** Open the **AVD Manager** and create a new virtual device. Start it.
 - **VS Code:** Open the command palette (Ctrl/Cmd+Shift+P), type Flutter: Launch Emulator, and select a device.
-

Step 2: Creating the Project & First Widgets

Now, let's create our project and build our first screen.

1. **Create a New Project:** In your terminal, navigate to the directory where you want to create your project and run:

```
Bash  
flutter create waiting_room_app
```
2. **Open the Project:** Open the new waiting_room_app folder in your IDE.
3. **Delete Boilerplate Code:** Open lib/main.dart and delete all the code, leaving only main() and a basic MaterialApp.

```
Dart  
import 'package:flutter/material.dart';  
  
void main() {
```

```

runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Waiting Room',
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Waiting Room'),
        ),
        body: const Center(
          child: Text('Welcome to the Waiting Room!'),
        ),
      ),
    );
  }
}

```

4. **Run the App:** Run the app on your emulator/device using the command:

```

Bash
flutter run

```

5. **Build the "Waiting Room Card":**

- Create a new file lib/waiting_room_card.dart.
- Create a StatelessWidget to display a name.

```

Dart
// lib/waiting_room_card.dart
import 'package:flutter/material.dart';
import 'package:waiting_room_app/waiting_room_timestamp.dart'; // Import the timestamp
widget

```

```

class WaitingRoomCard extends StatelessWidget {
  final String name;

  const WaitingRoomCard({super.key, required this.name});

```

```

@override
Widget build(BuildContext context) {
  return Card(
    margin: const EdgeInsets.all(16.0),
    child: Padding(
      padding: const EdgeInsets.all(24.0),
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          const Text(

```

```

        'Hello,',
        style: TextStyle(fontSize: 16),
      ),
      Text(
        name,
        style: const TextStyle(fontSize: 24, fontWeight: FontWeight.bold),
      ),
      const SizedBox(height: 10), // Add some spacing
      const WaitingRoomTimestamp(),
    ],
  ),
);
}
}

```

- Replace the Text widget in lib/main.dart with your new widget.

```

Dart
// In main.dart
// ...
import 'package:waiting_room_app/waiting_room_card.dart';

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      // ...
      body: const Center(
        child: WaitingRoomCard(name: 'John Doe'),
      ),
    );
  }
}

```

- Observe the change with Hot Reload.
- **Important:** You will need to create the waiting_room_timestamp.dart file. Refer to the **Annexe** at the end of this document for the complete code.

Step 3: Writing the First Widget Test

This step ensures your new widget works as expected.

1. **Open the Test Folder:** Navigate to test/. Create a new file test/waiting_room_card_test.dart.
2. **Write the Test:** Add the following code to the new test file.

```

Dart
// test/waiting_room_card_test.dart
import 'package:flutter_test/flutter_test.dart';

```

```
import 'package:waiting_room_app/waiting_room_card.dart';
import 'package:flutter/material.dart';

void main() {
  testWidgets('WaitingRoomCard displays the name correctly', (WidgetTester tester) async {
    // Build our widget.
    await tester.pumpWidget(
      const MaterialApp(
        home: WaitingRoomCard(name: 'Alice'),
      ),
    );

    // Verify that the widget renders the correct name.
    expect(find.text('Hello,'), findsOneWidget);
    expect(find.text('Alice'), findsOneWidget);
  });
}
```

3. **Run the Test:** In your terminal, run the test command.

```
Bash
flutter test test/waiting_room_card_test.dart
```

- You should see a message indicating All tests passed!

Step 4: Git Workflow & Pull Request

Now, you will formalize your work using the team's Git workflow.

1. **Initialize Git & Create a Repo:**

- In your terminal, navigate to the project root and run:


```
Bash
git init
git add .
git commit -m "feat: initial project setup"
```
- Go to GitHub or GitLab, create a new public/private repository, and follow the instructions to link your local project to it.


```
Bash
git remote add origin <your-repo-url>
git branch -M main
git push -u origin main
```

2. **Create a Feature Branch:**

```
Bash
git checkout -b feature/waiting-card-refactor
```

3. **Modify and Commit:** Go back to your code and add the timestamp StatefulWidget to the card.

- Create lib/waiting_room_timestamp.dart (refer to the Annexe).
- Modify lib/waiting_room_card.dart to include the timestamp widget.
- Commit your changes with a meaningful message:

```
Bash
git add .
git commit -m "feat: add live timestamp to waiting room card"
```

4. Push and Open a PR:

```
Bash
git push -u origin feature/waiting-card-refactor
```

- Go to your repository on GitHub/GitLab. You will see a prompt to open a Pull Request. Fill in the title and description clearly.

Step 5: Setting Up Basic CI

This step automates testing on every PR.

- Create the CI File:**
 - In the project root, create a new directory: .github/workflows/.
 - Inside this directory, create a file named ci.yml.
- Add the CI Configuration:** Copy and paste the following code into the ci.yml file.

YAML

```
name: Flutter CI
```

```
on:
```

```
  pull_request:
```

```
    branches: [ main ]
```

```
  push:
```

```
    branches: [ main ]
```

```
jobs:
```

```
  build:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - uses: actions/checkout@v2
```

```
      - uses: subosito/flutter-action@v2
```

```
      with:
```

```
        flutter-version: '3.19.0' # Use your Flutter version here
```

```
      - name: Install dependencies
```

```
        run: flutter pub get
```

```
      - name: Run tests
```

```
        run: flutter test
```

```
      - name: Analyze code
```

```
        run: flutter analyze
```

3. **Commit and Push:** Add the new .github directory, commit, and push it to your feature branch.

```
Bash
git add .github/
git commit -m "ci: add basic CI pipeline for PRs"
git push
```

- Your CI job will now run automatically on your PR. Wait for it to pass.

4. **Merge the PR:** Once your CI job passes and you've received a code review, you can merge your branch into main.

Deliverables Checklist

Before the workshop concludes, ensure you have completed the following:

- [] A private repository with your project.
- [] The project runs successfully with flutter run.
- [] A passing widget test is present in the test/ directory.
- [] You have a merged PR in your repository.
- [] The CI pipeline successfully runs flutter test and flutter analyze on your merged PR.

Annexe: waiting_room_timestamp.dart

Dart

```
import 'package:flutter/material.dart';
import 'dart:async'; // Required for the Timer class

class WaitingRoomTimestamp extends StatefulWidget {
  const WaitingRoomTimestamp({super.key});

  @override
  State<WaitingRoomTimestamp> createState() => _WaitingRoomTimestampState();
}

class _WaitingRoomTimestampState extends State<WaitingRoomTimestamp> {
  // A variable to hold the current time
  late DateTime _currentTime;

  // A timer to update the time periodically
  late Timer _timer;

  @override
  void initState() {
    super.initState();
    // Initialize the current time when the widget is created
    _currentTime = DateTime.now();
  }
}
```

```

// Set up a periodic timer to update the state every second
_timer = Timer.periodic(const Duration(seconds: 1), (timer) {
  // Use setState to rebuild the widget with the new time
  setState() {
    _currentTime = DateTime.now();
  });
});
}

@override
void dispose() {
  // This is crucial! Cancel the timer when the widget is removed from the tree
  // to prevent memory leaks and unnecessary processing.
  _timer.cancel();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  // Format the date and time to a readable string
  final formattedTime = _currentTime.toString().split(':')[0];

  return Text(
    'Current Time: $formattedTime',
    style: const TextStyle(
      fontSize: 14,
      color: Colors.black54,
    ),
  );
}
}

```